

QUESTION 2

Title: Evaluate Quick Sort Performance with Adaptive Recursion Limits to Prevent Stack Overflow

Aim:

To evaluate Quick Sort performance using adaptive recursion limits, measure execution time and recursion depth, and analyze how limiting recursion improves stability and prevents stack overflow.

Problem Understanding:

Quick Sort recursively partitions an array around a pivot. In the worst case, recursion depth may become $O(n)$, leading to stack overflow for large inputs.

Adaptive recursion limits help control stack growth and improve stability by switching to safer processing methods when recursion becomes too deep.

Algorithm:

1. Select pivot element.
2. Partition array.
3. Recursively sort smaller partition first.
4. Track recursion depth.
5. Measure execution time.
6. Compare performance and stability.

Pseudocode:

QUICKSORT(arr, low, high, depth)

Update maximum depth

WHILE low < high

Partition array

Recursively sort smaller partition

Process larger partition iteratively

RETURN

Code:

```
import random
import time
max_depth = 0

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high, depth=1):
    global max_depth
    while low < high:
        max_depth = max(max_depth, depth)
        pi = partition(arr, low, high)
        if (pi - low) < (high - pi):
            quick_sort(arr, low, pi - 1, depth + 1)
            low = pi + 1
        else:
            quick_sort(arr, pi + 1, high, depth + 1)
            high = pi - 1
    sizes = [1000, 5000, 10000]
    for size in sizes:
        arr = [random.randint(1, 100000) for _ in range(size)]
        max_depth = 0
```

```
start = time.time()
quick_sort(arr, 0, len(arr) - 1)
end = time.time()
print("\nInput Size:", size)
print("Execution Time:", round(end - start, 6), "seconds")
print("Maximum Recursion Depth:", max_depth)
```

Sample Output:

Input Size: 1000

Execution Time: 0.0025 seconds

Maximum Recursion Depth: 8

Input Size: 5000

Execution Time: 0.0141 seconds

Maximum Recursion Depth: 11

Input Size: 10000

Execution Time: 0.0298 seconds

Maximum Recursion Depth: 12

Analysis:

Without Adaptive Recursion:

Worst Case: Recursion Depth = $O(n)$

Example: Already Sorted Array

May cause stack overflow.

With Adaptive Recursion:

The algorithm recursively processes the smaller partition first and iteratively handles the larger partition.

Benefits:

- Reduced recursion depth
- Lower stack usage
- Better stability
- Less risk of stack overflow

Practical Implementation Strategies:**1. Sort Smaller Partition First**

Maintains recursion depth near: $O(\log n)$

2. Random Pivot Selection

Avoids worst-case partitions.

3. Median-of-Three Pivot

Produces balanced partitions.

4. Hybrid Quick Sort

Switch to Insertion Sort for small subarrays.

Time Complexity:

Best Case : $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n^2)$

Space Complexity: $O(\log n)$

With adaptive recursion.

Result:

The experiment demonstrates that adaptive recursion significantly reduces Quick Sort recursion depth and prevents stack overflow. Processing the smaller partition recursively keeps stack usage close to $O(\log n)$, improving stability and making Quick Sort suitable for large datasets on memory-constrained systems.